

IA Aplicada com LLMs

Aula 05 — Padrões de arquitetura de agentes

Onde paramos

Na aula passada vocês leram uma dúzia de casos perguntando "*que padrão é este?*" — e responderam com **nomes**.

E a Aula 03 tinha deixado uma lista:

estado e memória · múltiplas ferramentas · outros padrões de arquitetura · MCP · confiabilidade · context engineering dinâmica

Hoje os nomes viram código, e riscamos quatro da lista.

Vocês têm um laço que funciona

```
for passo in range(max_passos):  
    r = client.chat.completions.create(messages=mensagens, tools=DECLARACOES)  
    if not r.choices[0].message.tool_calls:  
        return r.choices[0].message.content  
    ...
```

Por que ele não está pronto para rodar sozinho?

1. **Não sabe parar** — `max_passos` é teto, não orçamento → nota 03
2. **Não sabe falhar** — uma exceção derruba tudo → nota 03
3. **Não sabe esquecer** — o histórico só cresce → nota 04

E uma quarta pergunta, anterior a todas

Ele precisava ser um laço?

Este deck é sobre essa pergunta.

E como vocês já têm um case escolhido, a pergunta em cada padrão não é "*entendi?*" — é "**é este que o meu case precisa?**"

Roteiro: a linha divisória · autópsia do exercício 03 ·
cinco padrões · a tabela de decisão · composição

Parte 1

A linha divisória

De definição a pergunta operacional

Workflow: o caminho está no código.

Agente: o caminho é decidido em execução pelo modelo.

Boa para prova. Ruim para decidir às três da tarde.

Dá para desenhar o fluxograma antes de rodar?

Sim → desenhe, e escreva o fluxograma em Python.

Não → aí existe motivo para autonomia.

Três perguntas de apoio

	Sim	Não
O nº de passos é conhecido?	workflow	agente
A ordem depende de dados de execução?	agente	workflow
O erro é caro ou irreversível?	reduza a autonomia	tolerável

A terceira não é sobre a tarefa — é sobre a **consequência do erro**.

Tarefa que pediria um agente, mas cujo erro emite dinheiro,
é tarefa onde se aceita um sistema pior para tê-lo previsível.

O que custa subir no espectro

Nível	Chamadas	Previsível	Depurável
Prompt único	1	sim	sim
Chain	N fixo	sim	sim
Roteador	1 + rota	sim	sim
Agente	?	não	só com trace
Multi-agente	??	não	muito difícil

A coluna que dói é a última: num agente a trajetória muda a cada execução — sem log você tem uma saída errada e nenhuma pista.

Parte 2

Autópsia do exercício 03

O melhor workflow disfarçado de agente que temos

...foi escrito por vocês, na semana passada.

Etapa	Veredito
1. Coleta	workflow — a parada é um <code>if</code>
2. Consulta	workflow — sempre, sempre nessa posição
3. Raciocínio	decisão — mas uma, não uma sequência
4. Ação	workflow condicional
5. Consulta final	workflow — sempre por último

Quatro das cinco etapas estavam no código

A sequência estava **numerada no enunciado**.

Foi um erro? Não — o objetivo era o **mecanismo**.

Mas em produção a versão workflow ganharia em tudo: menos chamadas, sem pular a consulta final, sem abrir dois chamados, testável etapa a etapa. Perderia só o atendimento fora do roteiro — e é aí que se decide.

Quando o enunciado vem numerado, o código também deveria vir.

Parte 3

Os cinco padrões

Building Effective Agents (Anthropic, 2024)

1. Prompt chaining com portão

```
entrada —> [ LLM 1 ] —> portão —> [ LLM 2 ] —> saída
                        |
                        └─ falhou —> não segue adiante
```

```
bruto = chamar(PROMPT_EXTRACAO, texto, schema=SCHEMA_EXTRACAO)
if not bruto.get("valor") or bruto["valor"] <= 0:    # 0 PORTÃO:
    raise ValorInvalido(bruto)                    # código, não modelo
return chamar(PROMPT_PARECER, bruto, schema=SCHEMA_PARECER)
```

Regra que cabe em Python vai em Python: mais barata, e não alucina.

Não usar: etapas interdependentes · nº de etapas que depende da entrada

2. Roteador — o modelo classifica, você despacha

```
entrada —> [ classificador ] —> rota A: código puro (sem LLM)
                                     ↳ rota B: outro prompt / outro modelo
                                     ↳ rota C: humano
                                     ↳ nenhuma —> fila de revisão
```

```
r = classificar(item, schema_com_enum(ROTAS))      # 1 chamada, barata
if r["rota"] == "dentro_da_politica":
    return aprovar_por_regra(item)                  # ZERO chamadas de LLM
if r["rota"] == "ambiguo": return agente_analisa(item)
if r["rota"] == "acima_da_alcada": return encaminhar_humano(item)
return fila_de_revisao(item)                        # "nenhuma": não invente
```

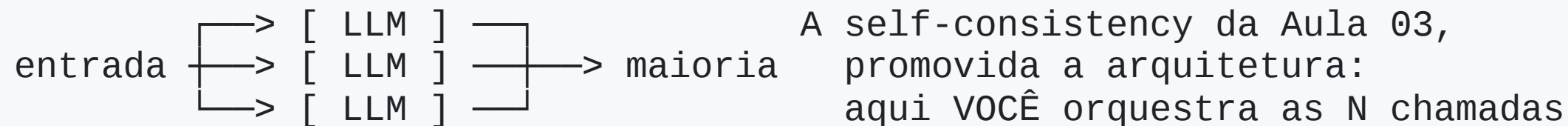
A rota **nenhuma** é obrigatória — sem escape o modelo escolhe uma opção que não serve, com confiança. **A rota mais valiosa é a que não chama o modelo:** se 70% caem em `aprovar_por_regra`, o sistema ficou 70% mais barato sem perder nada.

3. Paralelização — são duas coisas com o mesmo nome

Sectioning — subtarefas **independentes**, que você conhece



Voting — a *mesma* tarefa N vezes, maioria decide



3. Voting: a divergência é informação

```
vereditos = [classificar(item) for _ in range(n)]
rota, votos = Counter(v["rota"] for v in vereditos).most_common(1)[0]
if votos < n // 2 + 1:
    return {"rota": "nenhuma", "motivo": "sem maioria"}
```

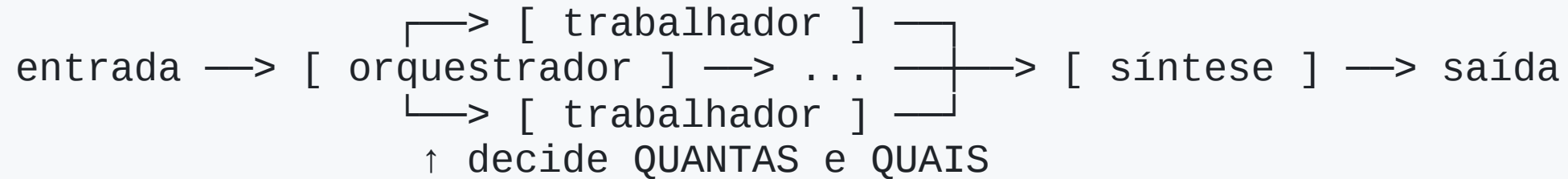
Três vereditos diferentes **não** significam "escolha um".

Significam: **este caso é difícil** — provavelmente é caso de humano.

É o sinal de incerteza mais barato que existe.

Não usar: *sectioning* com partes dependentes · *voting* em tarefa aberta ("escreva um resumo") — não há maioria entre três textos

4. Orquestrador-trabalhador

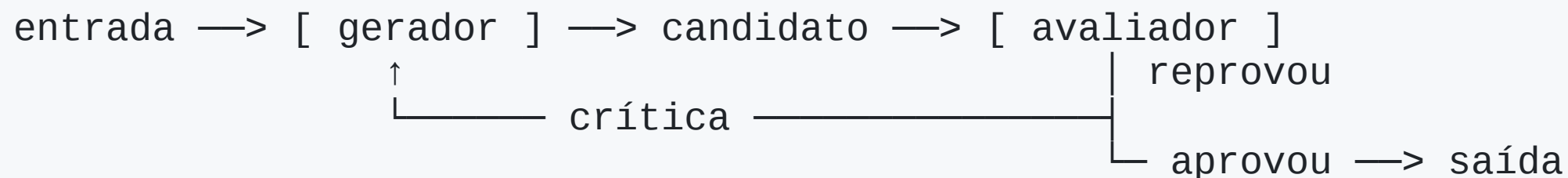


```
plano = chamar(PROMPT_ORQUESTRADOR, lote, schema=SCHEMA_PLANO)
if len(plano["subtarefas"]) > MAX_SUBTAREFAS:      # orçamento, já aqui
    raise PlanoGrandeDemais(len(plano["subtarefas"]))
resultados = [executar_subtarefa(s) for s in plano["subtarefas"]]
```

Diferença para o *sectioning*, e é toda a diferença:

as subtarefas só existem em execução. Sem teto, é uma conta aberta assinada por um modelo. **Na dúvida entre os dois: prefira sectioning.**

5. Avaliador-otimizador (Reflexion, Shinn et al. 2023)



```
for rodada in range(max_rodadas):
    candidato = chamar(PROMPT_GERADOR, entrada, critica=critica)
    aval = chamar(PROMPT_AVALIADOR, candidato, schema=SCHEMA_AVALIACAO)
    if aval["aprovado"]: return candidato, "aprovado"
    critica = aval["o_que_corrigir"]
return candidato, "teto_de_rodadas"      # melhor esforço, e AVISA
```

5. Duas armadilhas — e aparecem sempre

Sem critério escrito, o avaliador vira elogio.

```
{"nota": 8}                                # inútil  
{"cita_regra": false, "o_que_corrigir": "..."} # útil
```

Sem teto de rodadas, o par oscila. O avaliador pede A, o gerador entrega A e perde B, o avaliador pede B. E o retorno tem que **dizer** que saiu por teto — melhor esforço vendido como aprovado é pior que reprovação honesta.

Não usar: quando você não consegue escrever o critério.

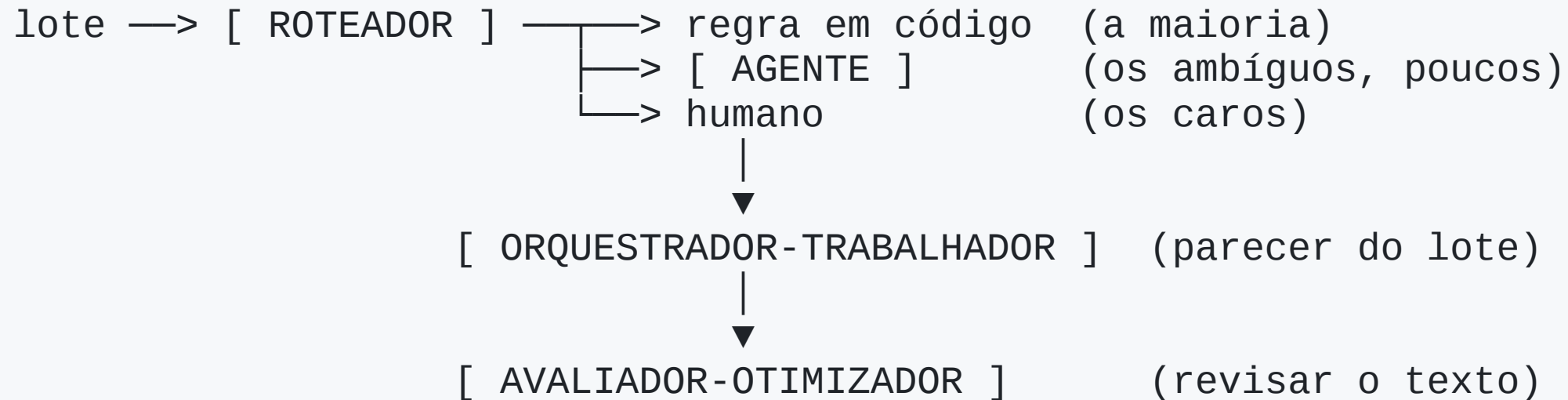
Aí o avaliador certo é uma pessoa.

A tabela de decisão

Padrão	Custo	Não usar quando
Prompt único	1	precisa de validação por outra chamada
Chaining + portão	N fixo	etapas interdependentes
Roteador	1 + rota	rotas quase iguais
Sectioning	1 por seção	partes dependentes
Voting	3–5	tarefa aberta
Orquestrador	? — exige teto	you consegue listar as subtarefas
Avaliador	2 por rodada	sem critério escrito
Agente	? — exige orçamento	o fluxograma existe

Leia de cima para baixo, **pare no primeiro que resolve.**

Os padrões se compõem



Este é o desenho do exercício de hoje: a autonomia cara fica confinada ao caminho estreito, e o volume passa por código determinístico.

O exemplo que fecha o deck

1.000 despesas, 15% violam a política:

Arquitetura	Conta	Chamadas
Agente para tudo	$1.000 \times \sim 4$ passos	~ 4.000
Chain fixa	$1.000 + 150$	1.150
Roteador + regra em código	$0 + 300 + 150$	450

Nove vezes menos — e os 70% resolvidos por regra são **mais** confiáveis.

O ganho não veio de prompt melhor nem de modelo melhor.

Veio de decidir **o que não mandar para o modelo**.

Síntese

- Antes de melhorar o laço: **a tarefa precisava de um laço?**
- A pergunta é uma: **dá para desenhar o fluxograma antes de rodar?**
- Subir de nível custa dinheiro, previsibilidade e **capacidade de depurar**
- No roteador, a rota mais valiosa é a que **não chama o modelo**
- *Sectioning* × orquestrador: **quem define as subtarefas?**
- Avaliador sem critério produz elogio; sem teto, o par oscila
- *Voting*: a divergência é sinal de incerteza — não descarte

Autonomia é recurso escasso. Gaste onde ela compra alguma coisa.